

An Event Model for C++ Executors

Doc. No.: P2882R0

Date: 2023-05-11

Reply to: Detlef Vollmann, dv@vollmann.ch

Audience: SG1, (LEWG)

Preface

This is (currently) a purely exploratory paper, it is not any kind of proposal!

The goal of this paper is to start a discussion and to find out whether SG1 (and WG21 as a whole) is willing to provide a communication mechanism for diverse C++ execution contexts.

And a note on the code in this paper: All code is “slide code”, i.e. it hasn’t even run through a compiler. The purpose of the code is to demonstrate specific issues, not to provide a specific solution.

Abstract

P2300 proposes a framework for handling asynchronous events and start work on different execution contexts. However it doesn’t provide a low-level interface how to communicate and synchronize between such execution contexts.

This paper explores some requirements and design options to provide a standard mechanism for synchronization across execution contexts and presents a list of questions that SG1 should answer.

Introduction

P2300 allows for different (heterogeneous) execution contexts, but it doesn't specify any requirements for such execution contexts except that they must provide a scheduler. This makes it impossible to even implement a `transfer()` algorithm as there's no interface to join an execution agent from a different execution context and no interface to exchange results between execution contexts.

In P0073R2 "On unifying the coroutines and resumable functions proposals" Torvald Riegel proposed an event class to synchronize between different execution contexts. While event never got a detailed discussion, it proposed the basic required interface: `block()` and `signal()`. This interface is probably sufficient for blocking synchronization (and we already have the atomic interface for non-blocking synchronization).

Example

As an example a simple network speaker implementation could look like this:

A receiver to receive network packets from a network:

```
SnapClient srv{srvAddr, srvPort};

while (true)
{
    std::span<uint8_t> buf = srv.receiveWireChunk(); // blocks
    opusQueue.wait_push(buf);
}
```

A decoder decodes these packets to PCM frames:

```
while (true)
{
    std::span<uint8_t> inBuf;
    opusQueue.wait_pop(inBuf);
    int samples = opus_decode(decoder,
                              inBuf.data(), inBuf.size(),
                              decodeBuf.data(), maxFrameSamples,
                              0);
    std::span outBuf(decodeBuf.data(), samples);
}
```

```

        pcmQueue.wait_push(outBuf);
    }

```

And a play function sends the PCM frames to an I2S interface:

```

while (true)
{
    std::span<uint8_t> inBuf;
    pcmQueue.wait_pop(inBuf);
    uint32_t const *start = inBuf.data();

    size_t offset = 0;
    while (offset < size)
    {
        size_t bytesDone;
        // i2s_channel_write blocks
        i2s_channel_write(tx,
                           start + offset,
                           size - offset,
                           &bytesDone,
                           noTimeout);
        offset += bytesDone;
    }
}

```

These three functions could run in three separate threads (e.g. on a non-static thread pool).

Another solution might use the sender/receiver framework:

```

exec::scheduler auto sched0 =
    cpos::ContextCore0::LoopScheduler();
exec::scheduler auto sched1 =
    cpos::ContextCore1::LoopScheduler();

AudioServer srv(sched0, srvAddr, audioPort);

srv.readPacket()
    | bufferedTransfer<netBufferSize>(sched1)
    | then(soundDecode)
    | bufferedTransfer<pcmBufferSize>(sched0)
    | sendI2sChunk()
    | runForever();

```

Here bufferedTransfer uses a queue internally.

Yet another solution might use coroutines:

```
CoHandle receive()
{
    while (true)
    {
        std::span<uint8_t> buf = co_await srv.coReceiveWireChunk(); //
        blocks
        co_await opusQueuePush.wait_push(buf);
    }
}

CoHandle decode()
{
    while (true)
    {
        std::span<uint8_t> inBuf;
        co_await opusQueuePull.wait_pop(inBuf);
        int samples = opus_decode(decoder,
                                   inBuf.data(), inBuf.size(),
                                   decodeBuf.data(), maxFrameSamples,
                                   0);
        std::span outBuf(decodeBuf.data(), samples);
        co_await pcmQueuePush.wait_push(outBuf);
    }
}

CoHandle play()
{
    while (true)
    {
        std::span<uint8_t> inBuf;
        co_await pcmQueuePull.wait_pop(inBuf);
        uint32_t const *start = inBuf.data();

        size_t offset = 0;
        while (offset < size)
        {
            size_t bytesDone;
            co_await co_i2s_channel_write(tx,
```

```

        start + offset,
        size - offset,
        &bytesDone,
        noTimeout);

    offset += bytesDone;
}
}
}

```

A solution using fibers would essentially look like the threads version.

Currently Standard C++ only provides mechanisms for the threads version (though the queue is still missing). But as coroutines are already standardized and fibers on their way there should be a more generic mechanism.

Some Details

Implementing a queue that works for the different solutions is a challenge. Implementing a queue that only supports coroutines, or only fibers, or only threads is quite simple.

But if the solution with coroutines turns out to be too slow and the decoding should be done on a different core, while `receive()` and `play()` remain coroutines, it gets more difficult.

And the queues used here are completely single-ended, but e.g. for a mutex that works across all kinds of execution contexts it gets really complicated.

The event abstraction turns out to be useful. By implementing such an event class across different execution context it turns out that it's probably sufficient for `block()` to know about the current (blocking) execution context. If the current execution context is known statically, that's no problem. But if the current execution context is not known statically (e.g. when locking a mutex in some library class) there must be a mechanism to detect the current execution context at runtime.

But for `signal()` (or `notify()`) the execution context of the task to be notified must (quite probably) be known. For this the event object would need to keep a list of all tasks that are currently blocked on this event. It may be possible to avoid heap allocation by allocating the space for the respective data on the stack of the blocking function. But it should be sufficient for the `notify()` to know about the receiving context.

If this turns out to be true (i.e. `block()` only needs to know the current context and `notify()` only needs to know the receiving context(s)) it would avoid the m:n problem that David Goldblatt mentioned in a reflector thread.

There may be cases where for the `block()` it should be known if the respective notifier is on the same execution context (e.g. for fibers without scheduler), but even then it's not m:n. And in general it may not be known at blocking time who will do the `notify()` (e.g. for `condition_variable` or multi-ended queues).

Design Options

While experimenting with a queue implementation that works across execution contexts it turned out to be useful to split the queue into a body with no public interface and push and pull front ends that provided the push and pop functions, respectively. In this implementation the queue front ends were templated on the execution context. And the execution contexts provided a `block()` function that could be used by the queue front ends. This particular implementation didn't need to provide a `notify()` that was specific to the execution context, but in general this would be a requirement on the execution context as well.

This could be generalized to all kinds of synchronization mechanisms, but knowing the execution context statically is often infeasible.

Detecting the current execution context at runtime may be a challenge, but it may be possible.

Another design question is if heap allocation for these mechanism would be allowed or needs to be always avoided. Events need to keep a list of all tasks blocked on it, and this list is potentially unbounded.

And this list probably needs to be type erased, which again may require memory allocation.

In many cases some kind of event loop may be necessary. The Networking TS provided such an event loop (`io_service`). Again, to deal with all kinds of continuations (*completion tokens*) it required type erasure.

Questions

Before investing in more design explorations, SG1 should provide answers to a number of questions:

1. Should SG1 put more work into a generic synchronization interface?
2. Is event a usable abstraction?
3. Can we require an implementation to detect the top execution context a thread of execution is running on?
 - a. The full chain of execution contexts?
4. Do we need a list of potentially blocking functions in the standard library?
5. Do we want to allow customization for specific pairs of execution agents?